CodiComperChat Portfolio

어제의 이야기를 기억하는 캐릭터와 이어 가는 캠페인

기억과 관계가 쌓이는 캐릭터 캠페인을 기획부터 운영까지 구현한 1인 개발 프로젝트입니다. 실제 화면, 설계의 이유, 모델 평가와 배포 중 만난 문제를 함께 정리했습니다.

구조는 저장소 기준, 시험 수는 docs/plan/TESTING.md의 날짜별 기록입니다. 최근 전체 E2E에는 미해결 실패 3건이 있으며 이번 갱신에서 제품 전체를 재검증한 수치가 아닙니다.

기간	2026-09-15 ~ 2026-11-11 · 계속 개발 중
역할	1인 개발 (기획 · 프론트 · 백엔드 · AI · 배포)
서비스	https://codicomperchat-web.vercel.app
저장소	https://github.com/samam0234/CodiComperChat

규모	값
제품 애플리케이션	4
DB 테이블	38
DB 마이그레이션	36
API 시험 · 11월 11일 기록	249
AI 단위 시험 · 11월 11일 기록	92
E2E 사례 · 통과 현황 별도	123

목차

01 프론트엔드 — 이용자 웹 · Next.js 16

02 어드민 콘솔 — 운영자 웹 · Next.js 16

03 백엔드 서버 — 신뢰 경계 · Nest.js 12

04 AI 서비스 — 연기 · 판정 · 그림 · FastAPI

05 미래 지향 — 남은 일과 다음 단계

06 기획 스프레드시트 — 기획 · 요구사항 · 화면 · 일정

07 ERD — 테이블 38 · 외래 키 41

08 AI 가이드 챗봇 — LangGraph · RAG

09 개발 기록 — 최근 변경 · 운영 반영 상태

10 프레임워크 — 선택 이유 · 코드 · 경계

프로젝트 소개

유저가 주인공인 이야기를, 성격이 흔들리지 않는 AI 캐릭터가 함께 이어 가는 웹 서비스. 대화를 닫았다 다시 열어도 캐릭터가 지난 일을 기억하고, 관계가 쌓이고, 숨겨 둔 비밀과 부탁이 때가 되면 드러난다.

반복되던 문제

- 대화가 길어지면 앞일을 잊는다
- 물어보면 설정에 없는 개인사를 지어낸다
- 비밀 설정을 유도하면 털어놓는다
- 대화 말고는 할 게 적다

차별점을 모델이 아니라 '상태'에 두었다. 무엇을 기억하고, 무엇을 숨기고, 무엇을 코드가 대신 판단할지를 설계했다.

보통의 캐릭터 채팅	코디컴퍼챗
긴 대화가 쌓이면 앞일을 잊는다	기억 · 요약 · 친밀도 · 복선을 따로 저장하고 필요한 만큼만 넣는다
없는 개인사를 지어낸다	설정에 근거가 없으면 단정하지 않게 막고, 지어냈으면 다시 쓴다
비밀을 유도하면 털어놓는다	비밀을 모델에 아예 보내지 않는다
대화 말고는 할 게 적다	속마음 · 일기 · 문자 · 게시판 · 장면 그림 · 웹툰으로 넓어진다
캐릭터는 프롬프트 한 덩어리	8단계 카드와 3턴 테스트를 통과해야 발행된다
한 번에 한 캐릭터와만	월드코퍼챗: 한 세계관의 여러 인물과 파트 · 지도 · 미니게임으로

간단한 기능 소개

- 이어지고 쌓이는 대화 — 요약·친밀도·복선과 장기 기억을 분리합니다. 직접 기억을 적거나 지우고, 마지막 대사를 수정·삭제·재작성할 수 있습니다.
- 비밀 퀘스트 — 친밀도 문턱을 넘으면 캐릭터가 부탁을 꺼낸다. 달성은 모델이 아니라 코드가 인용을 대조해 판정한다.
- 월드코퍼챗 — 세계관 · 인물 · 장소 · 파트 · 지도로 여러 인물과 이야기를 잇는다. 미니게임 결과는 서버만 정한다.
- 캐릭터 공방 — 8단계 카드 + AI 초안 + 발행 전 3턴 테스트(말투 붕괴 · 비밀 누설 판정).
- 코디포토넷 · 웹툰 — 줄거리를 읽고 장면을 설계해 그린다. 모든 그림은 검수를 지나야 돌아오고, 막히면 코레딧 환불.
- 운영과 보안 — 이용자 문의와 운영자 답변, 신고·검수·계정 관리. AI 대기열은 대화 요청을 먼저 처리하고 실패한 생성은 환불합니다.

기술 스택

이용자 웹	Next.js 16 · React 19 · TypeScript · CSS Modules
운영 콘솔	Next.js 16 (웹과 쿠키 · 배포 분리)
백엔드	Nest.js 12 (ESM) · Prisma 7 · PostgreSQL 16
AI 서버	FastAPI · Python 3.12 · Ollama · diffusers

대화 모델	기본 gemma4:e4b(로컬) · 강화 gemma-4-31b-it(키 있으면 클라우드) · 예비 gemma4:e4b
시험	Vitest · Playwright · unittest · 자체 평가 스크립트
인프라	Docker Compose · Caddy · GitHub Actions · Vercel · Oracle Cloud



시스템 구조

01 · FRONTEND

프론트엔드 — 이용자 웹

화면 36개의 Next.js 16 이용자 사이트. 판단은 하지 않고, 읽기는 서버 컴포넌트, 쓰기는 서버 액션으로 Nest만 부른다. 세션 토큰은 브라우저가 모른다. 1:1 답만 일회용 표로 Nest 웹소켓에 붙고, 그 소켓은 글을 옮길 뿐 추론을 나누지 않는다.

01. 개요

이용자가 캐릭터를 둘러보고, 대화하고, 공방에서 캐릭터를 만들고, 월드코퍼챗을 플레이하는 웹이다. 화면은 35개, 묶음은 둘러보기 · 대화 · 오퍼레이션 · 공방 · 월드 · 내 것 · 계정 · 요금 · 안내 아홉 갈래다.

- 서버 컴포넌트가 lib/*.ts(server-only)로 읽고, 클라이언트는 app/actions/*.ts 서버 액션으로 쓴다
- 로그인 토큰은 httpOnly 쿠키에만 있고 서버 코드가 Authorization 헤더로 붙인다 → CORS를 열지 않는다
- 운영 배포: Vercel(icn1), API는 Oracle VM의 Caddy 뒤

02. 사용 엔진

프레임워크	Next.js 16 (App Router · 서버 컴포넌트 · 서버 액션)
UI	React 19 · TypeScript · CSS Modules
스타일	디자인 토큰(app/globals.css) — 라이트 · 다크, 한국 시각에 따라 자동 전환
보안	proxy.ts가 요청마다 nonce CSP 발급, 외부 스크립트는 결제창 하나뿐
시험	Playwright E2E 123개(운영 빌드 + 가짜 LLM · 가짜 그림 엔진)

03. 상세 기능

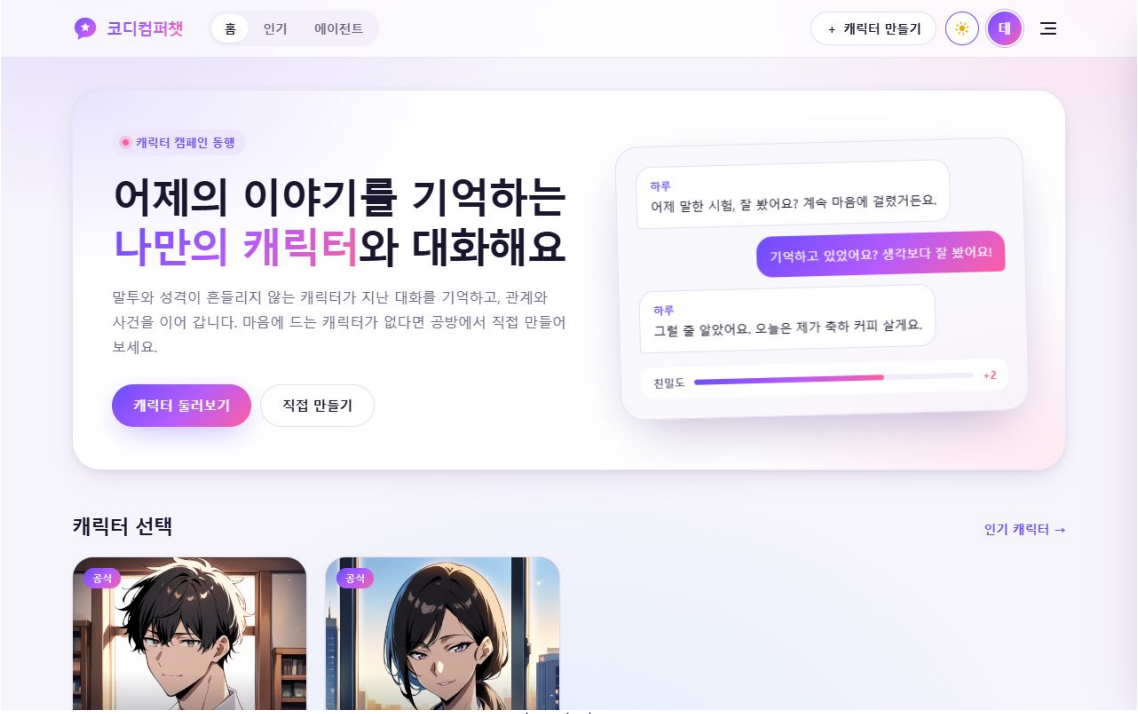
- 대화: 기억 · 지난 이야기 · 상태 설정 · 엑스트라 · 모델 고르기 · /다시 /계속 /되돌리기 명령어 · *행동* 과 @: 나레이션 꾸미기 · 대답 추천
- 오퍼레이션: 심리 탐구(속마음) · 일기 · 문자 · 가상 게시판 · 웹툰
- 공방: 8단계 카드 편집기, 칸마다 AI 초안, 3턴 테스트, 초상(파일 · AI)
- 월드코퍼챗: 세계관 편집기(인물 · 장소 · 파트 · 지도), 대화 방식 4가지(채팅 · 비주얼노벨 · 소설 · 메신저), 미니게임 창
- 글은 HTML로 넣지 않는다: lib/chat-format.ts가 표기를 읽고 React 요소로만 그린다(XSS 차단)
- 마지막 대사 수정·삭제·변경사항 추천으로 다시 쓰기
- 직접 적는 장기 기억과 대화 중 자동 사실 수집
- 대화·세계관 상단에 현재 시간과 장소 표시

04. API 연결

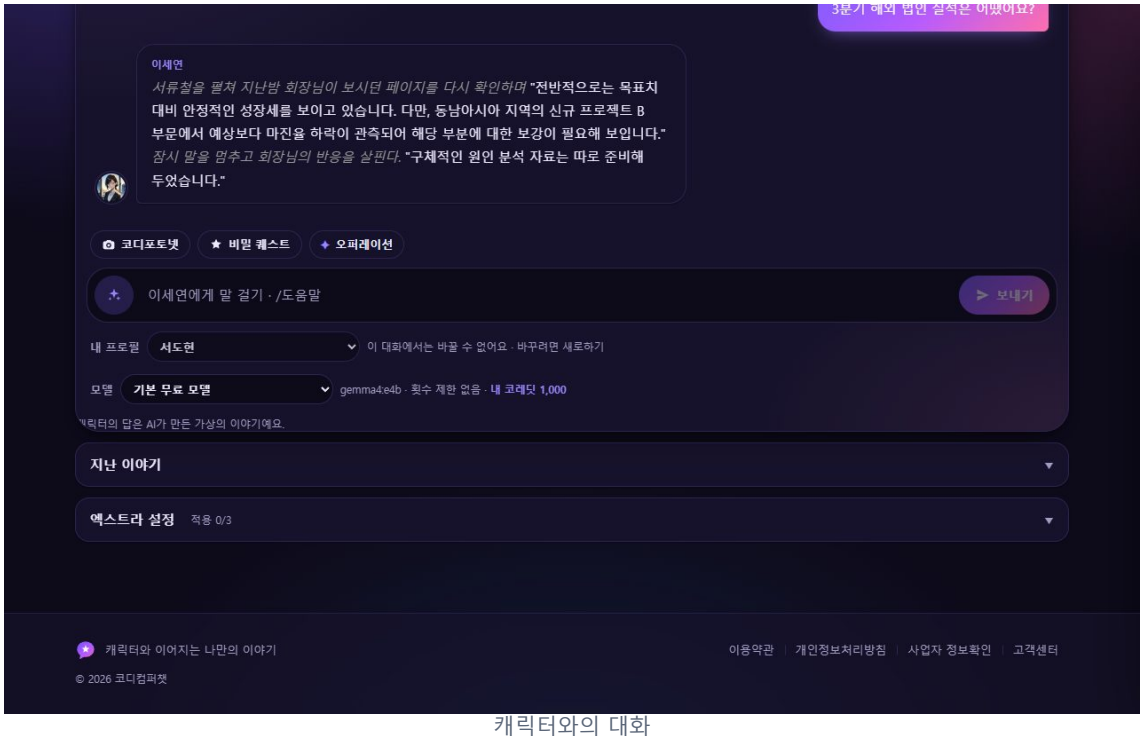
브라우저 → Next 서버 → Nest 순서만 허용한다. 브라우저에서 직접 부르는 길은 app/api/* 중계 라우트뿐이고, 여기에는 무거운 추론을 넣지 않는다.

읽기	서버 컴포넌트 → lib/*.ts → GET Nest (Bearer)
쓰기	클라이언트 → app/actions/*.ts(서버 액션) → POST Nest
대화 대기	app/api/characters/[id]/chat — 답을 기다리는 동안 대기 표시를 그리려고 쿠키 토큰을 붙여 넘긴다
그림	app/api/.../portrait · photonet · webtoon — <img>는 토큰을 못 붙여 서버가 받아 no-store로 돌려준다
이용자 IP	lib/client-ip.ts가 X-Forwarded-For + 프록시 토큰으로 넘긴다 → 요청 한도가 사람별로 센다

05. 내부 페이지



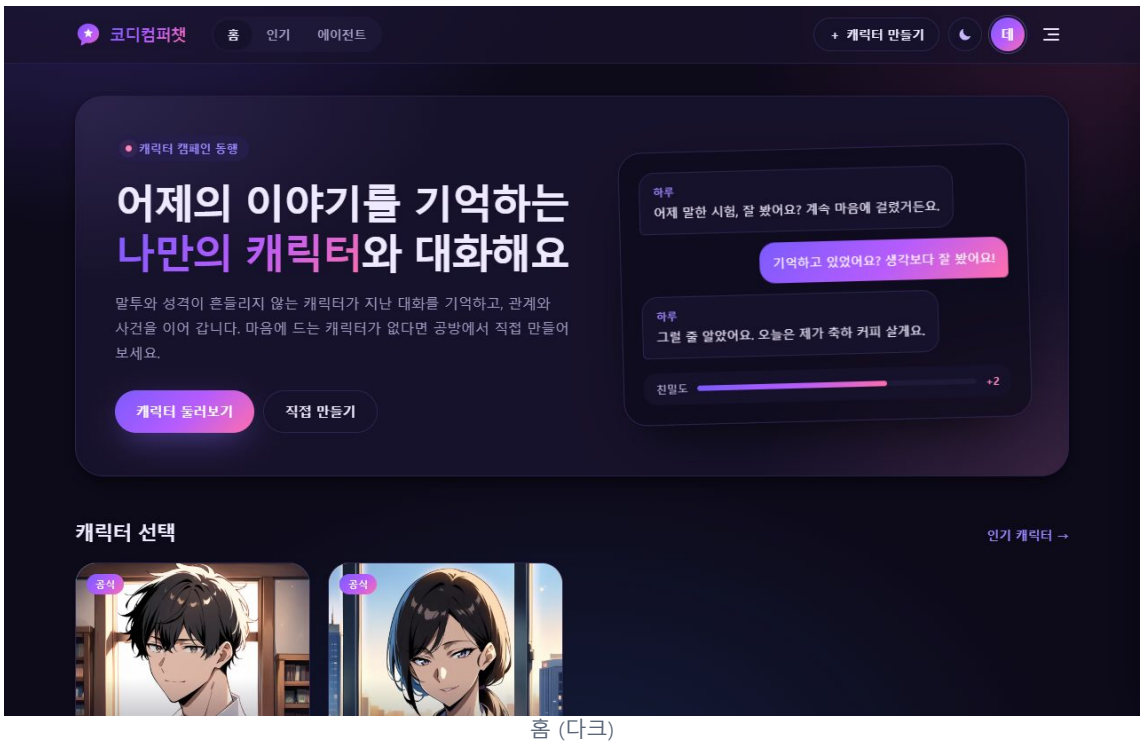
홈 (라이트)



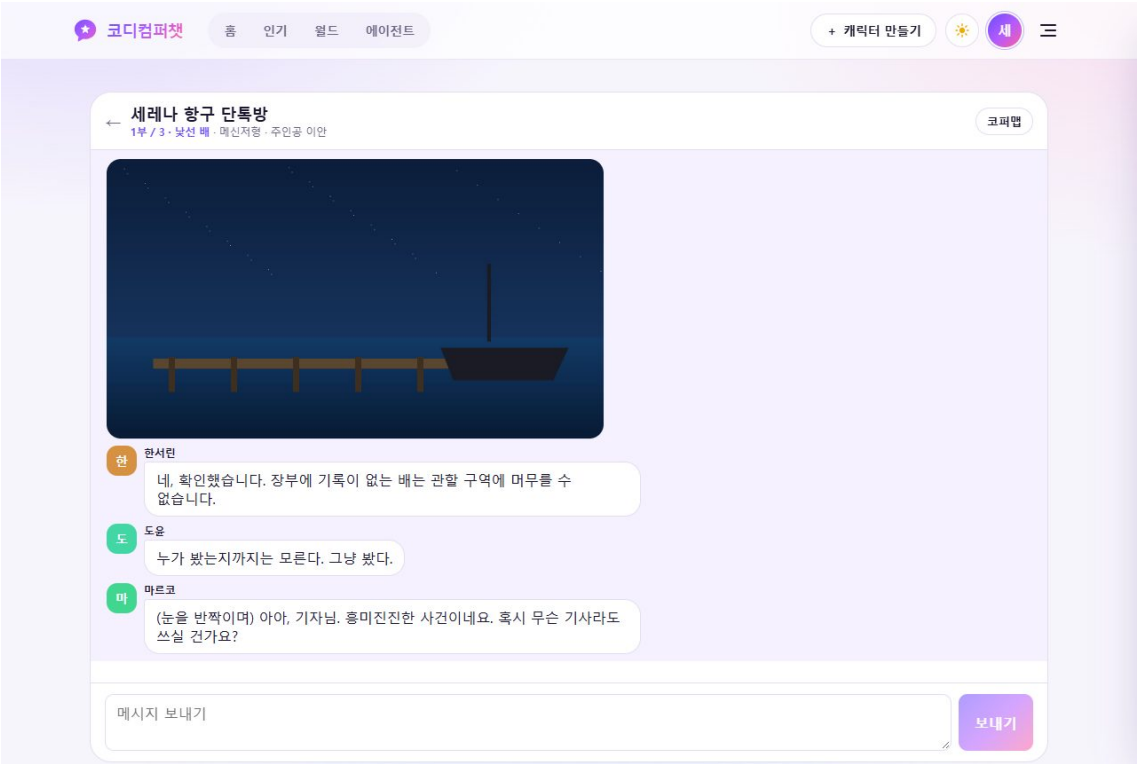
캐릭터와의 대화

06. 디자인

따뜻한 종이 · 구리 톤의 토큰 위에 라이트 · 다크 두 벌을 유지한다. 색 · 그림자 · 둥글기는 토큰만 쓰도록 규칙을 정해 테마가 깨지지 않게 했다. 대화는 말풍선 · 나레이션 · 행동 표기를 시각적으로 구분하고, 월드는 대화 방식마다 전혀 다른 레이아웃(무대 + 대사 상자, 문단, 메신저 말풍선)을 쓴다.



홈 (다크)



메신저형 월드

07. 문제점과 개선

좁은 화면(390px)	대화의 모델 고르기 칸이 눌려 글자가 안 보인다 → 고칠 예정(TASKS Q07)
응답 대기	1:1은 로컬·호스팅 모델의 토큰을 웹소켓으로 전달한다. 11월 11일 개선 코드는 병합됐지만 운영 API·AI 재배포와 실측은 아직이다. 월드는 완성 후 응답한다.
하이드레이션	toLocaleString 불일치 → lib/format-time.ts로 날짜 표기 통일(해결)
e2e 무작위 실패	next dev의 메모리 압박 → 운영 빌드로 시험(해결)

08. 미래 지향

- 월드 플레이도 1:1처럼 토큰을 바로 보여 주기
- 접근성 감사(키보드 · 스크린리더) 자동화
- PWA로 모바일 홈 화면 설치

02 · ADMIN CONSOLE

어드민 콘솔 — 운영자 웹

운영자 전용 Next.js 16 앱(화면 13개). 이용자 웹과 쿠키 · 배포를 나눠, 이용자 사이트에 문제가 생겨도 운영 도구가 같이 흔들리지 않게 했다.

01. 개요

신고 처리, 초상 · 그림 검수, 계정 정지와 등급 변경, 세계관 내리기와 이벤트 기록, 운영 설정을 한 곳에서 한다. 권한은 세 겹으로 확인한다.

- proxy.ts: 쿠키가 없으면 로그인으로
- 각 페이지의 requireAdmin(): Nest에 토큰을 확인, 역할 ADMIN
- Nest AdminGuard: DB의 역할을 다시 확인
- 공개 호스트는 CONSOLE_IP_ALLOWLIST에 있는 IP만 연다

02. 사용 엔진

프레임워크	Next.js 16 (웹과 같은 판, 별도 앱)
인증	콘솔 전용 쿠키, 운영자 계정만 로그인
보안	nonce CSP, X-Robots-Tag: noindex, IP 허용 목록
배포	Vercel 프로젝트 codicomperchat-console (API 연결은 허용 IP 결정 대기)

03. 상세 기능

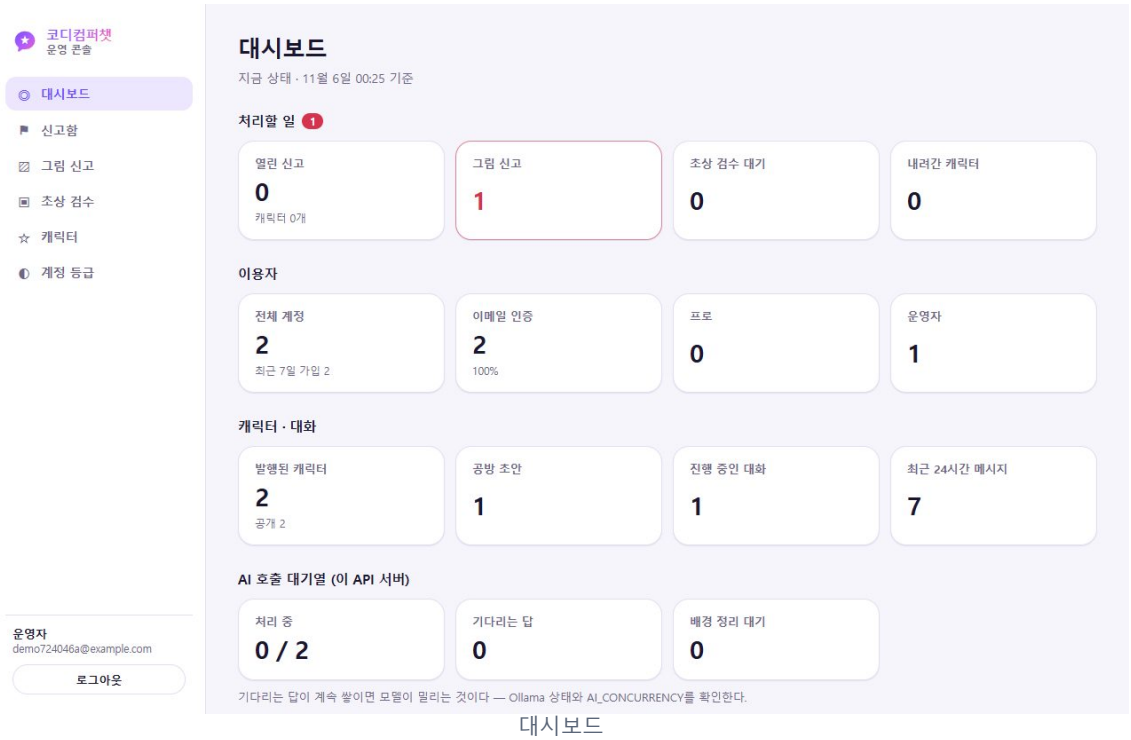
/	대시보드: 처리할 일 · 이용자 · 캐릭터 · AI 대기열
/reports	신고함: 내리기(사유 필수) · 기각 · 복구
/inquiries	고객 문의: 읽고 답하면 고객센터에 다시 보인다
/portraits	초상 검수: 자동 검수가 넘긴 것 승인 · 반려
/content	그림 신고: 접근 차단 · 계정 정지 · 해제
/characters	비공개 · 내려간 것까지 찾기 · 내리기 · 복구
/users	등급 변경, 결제 내역 · 환불, 생년월일 정정
/worlds/*	세계관 목록 · 상세(숨긴 파트 목표) · 검수 · 이벤트 기록 · 운영 설정

04. API 연결

콘솔도 Nest만 부른다(FastAPI 직접 호출 금지). 처리는 app/actions/admin.ts 서버 액션이 Nest /admin/* 로 넘기고, 그림은 app/api/... 중계가 검수 대기 · 막힌 그림까지 no-store로 가져온다.

GET /admin/stats	대시보드 숫자
POST /admin/characters/:id/take-down	캐릭터 내리기(사유 필수)
POST /admin/content-reports/:id/block	그림 접근 차단(+계정 정지)
POST /admin/orders/:id/refund	전액 환불(결제사 승인 뒤 기록)
GET·PUT /admin/world-settings	하루 보상 횟수 · 이벤트 확률 · 간격

05. 내부 페이지



대시보드

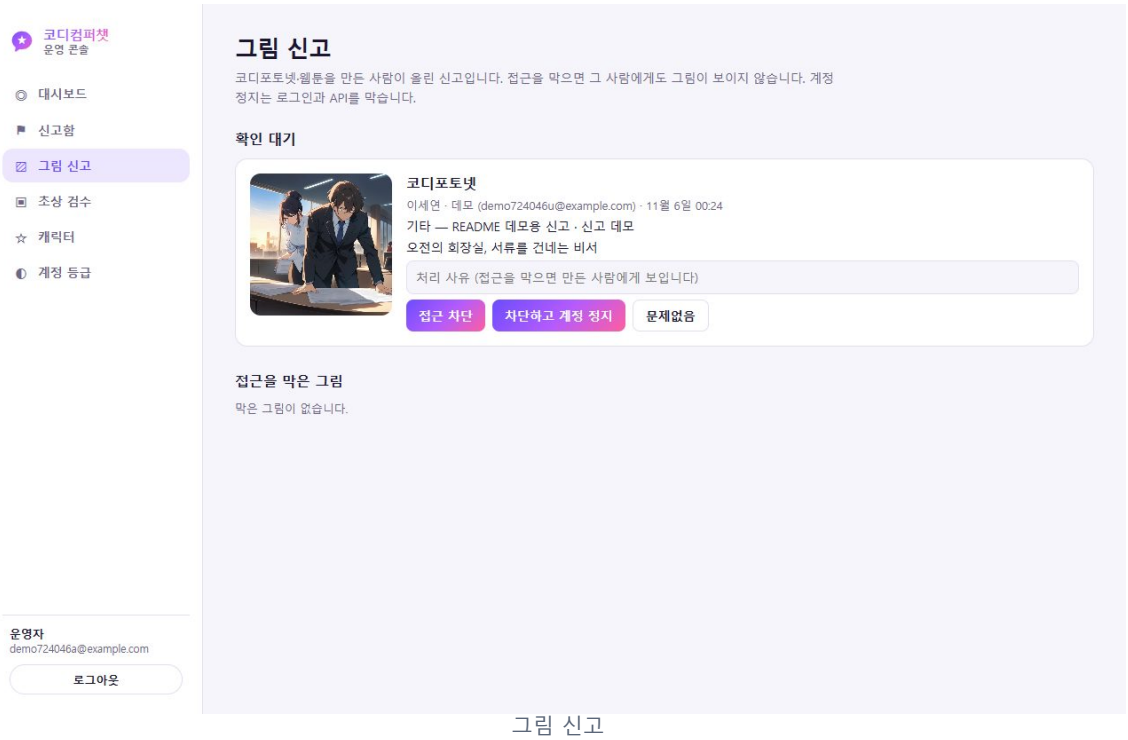


그림 신고

06. 디자인

이용자 웹과 같은 토큰을 쓰되 정보 밀도를 높였다. 숫자 줄 → 찾기 · 거르기 → 표 → 상세의 순서를 모든 화면에서 같게 해, 처음 보는 운영자도 같은 동작으로 일을 처리한다. 되돌릴 수 없는 처리(내리기 · 차단 · 환불)는 모두 사유를 받는다.

07. 문제점과 개선

운영 배포 연결	허용 IP 결정 전이라 운영 API에 연결하지 않았다(TASKS U08)
IP 거르기	Vercel에는 앞단 Caddy가 없어 앱 한 겹뿐 → 연결 전 헤더 신뢰 확인 필요
감사 로그	처리 기록은 DB에 남지만 별도 감사 화면이 없다

08. 미래 지향

- 운영자 처리 감사 로그 화면
- 신고 · 검수 큐의 알림(메일 · 웹훅)
- 역할 세분화(검수자 · 결제 담당)

03 · BACKEND

백엔드 서버 — 유일한 신뢰 경계

Nest.js 12(ESM) + Prisma 7 + PostgreSQL 16. 외부에 열린 유일한 백엔드로 인증 · 권한 · 소유 확인 · 요청 한도 · 코레딧 · 맥락 조립을 모두 맡는다. 판단 로직은 순수 함수로 모아 DB 없이 시험한다.

01. 개요

웹 · 콘솔은 Nest만 알고, Nest만 DB와 AI를 알고, AI는 DB를 모른다. 이 원칙 하나로 보안 검토와 시험 범위를 Nest 한 곳에 모았다. 모듈 30여 개, 2026-11-07에 대조한 API 경로 155개에 고객센터 문의 5경로를 더했다. 테이블 38개.

- Controller → Guard(Auth · Admin · RateLimit) → Service → 순수 함수 / Prisma / ai-client / Credits
- 운영 기동 검사: NODE_ENV=production에서 REQUIRE_HTTPS · 토큰 길이 · 개발 스위치를 보고 어긋나면 시작하지 않는다
- 운영: Oracle Cloud A1.Flex(ARM) Docker Compose — Caddy · api · postgres · ai · ollama

02. 사용 엔진

런타임	Node.js 24 · Nest.js 12 (ESM)
ORM · DB	Prisma 7 · PostgreSQL 16 · 마이그레이션 36개
메일	Nodemailer (SMTP · 폴더 · 로그)
결제	토스페이먼츠 연동(보류) · 눌러서 프로(PRO_CLICK)
시험	Vitest 249개 · 2026-11-11 기록. 실제 PostgreSQL 시험과 순수 함수 시험을 분리한다.

03. 상세 기능

character-chat	대화 한 턴: 맥락 조립 → /infer → 개인사 검사 → 저장
campaign-state	뒤에서 정리: 메시지 6개마다 요약 · 기억 · 친밀도 · 복선 · 퀘스트 판정
quests	비밀 퀘스트: 건넴 → 달성 → 보상 장면 한 번
worlds	월드코퍼챗: 세계관 · 플레이 · 3턴마다 파트 판정 · 미니게임 · 하루 보상 한도
studio	공방: 초안 → 3턴 테스트 → 발행(이메일 인증 계정만)
credits	코레딧: 출석 · 차감 · 실패 환불 · 보상을 DB 한 문장으로
persona-rag	2-gram BM25 + 키워드 보너스 지식 조각 검색(벡터 DB 없음)
대화 편집·기억	마지막 대사만 편집, 직접 기억 관리, notice-facts로 사실 자동 수집
support-inquiries	이용자 문의를 저장하고 콘솔 답변을 고객센터에 표시

04. API 연결

Nest 컨트롤러 경로 155개를 문서와 하나씩 대조해 빠진 것이 없음을 확인했다(2026-11-07). 2026-11-08에 고객센터 문의 5경로를 더했고, 전체를 다시 세지는 않았다. AI 호출은 대기열을 지나며, 화면에서 기다리는 답(foreground)이 뒤에서 하는 정리(background)보다 먼저 간다.

POST /auth/signup · /auth/login	가입 · 로그인 → 토큰(쿠키)
POST /characters/:id/chat	대화 한 턴 (답 · 사용량)
POST /worlds/:id/play/turn	월드 한 턴 (진행자 답 · 위치 · 이벤트)
POST /studio/characters/:id/test	발행 전 3턴 테스트
GET /health	상태 + AI 대기열(실행 · 대기 · 동시 수)

05. 내부 구조



시스템 구조



대화 한 턴의 처리 순서

06. 고급 기술

- 모델의 자기 확신을 믿지 않는다: 퀘스트 달성은 확신 ≥85, 인용 ≥8자, 인용이 그 화자의 실제 대사 안에 있을 때만 → 잘못 통과 0건
- 환각 억제: 설정에 없는 개인사를 묻는 턴만 /ground-check, 지어냈으면 다시 쓰기(최대 2번)
- AI 대기열은 foreground 우선. X-AI-Priority로 배경 작업을 구분하고, 호스팅 장애 때 배경 정리가 CPU 예비 자리를 차지하지 않게 했다(11월 11일 코드, 서버 반영 대기).
- 돈과 횟수는 DB 한 문장(원자적 UPDATE)으로 세서 동시 요청에도 두 번 깎이지 않는다
- 웹과 API가 다른 기계일 때 프록시 토큰으로만 이용자 IP를 믿는다

07. 문제점과 개선

호스팅 장애	기본은 로컬 e4b, 강화는 키가 있으면 호스팅 31B. 키·접근 오류에도 예비로 전환하는 코드를 추가했다. 운영 로그로 근본 원인은 미확인.
성인 모드 가드	호스팅 API 서버에서 성인 모드를 코드로 막는 장치가 없다(T05)
메일	운영 SMTP 미정 → 인증 메일 미발송
백업	운영 DB 백업 cron 미설정(T04)

08. 추가할 백엔드 서버

- GPU 서버(Hetzner GEX45)로 그림 엔진 이전, ai · ollama만 분리 가능하게 설계됨
- 여러 대로 늘릴 때 요청 한도 저장소를 DB로(RATE_LIMIT_STORE=db)
- 결제 웹훅 수신 서버(토스)

09. 미래 지향

- 월드 플레이 토큰 전달
- 관측성: 요청 · AI 대기열 지표 대시보드
- 읽기 복제본과 백업 자동 복구 연습

04 · AI SERVICE

AI 서비스 — 연기 · 판정 · 그림

FastAPI(Python 3.12) 내부망 전용 서버. DB를 보지 않고 Nest가 조립해 넘긴 조각만 쓴다. 자체 가중치를 학습하지 않는 대신, 프롬프트 · 맥락 · 판정을 실제 모델로 재고 고쳤다.

01. 개요

연기(/infer), 요약 · 기억(/summarize · /remember), 환각 판정(/ground-check), 월드 진행자(/world/turn · /world/part-check), 그림(/photonet · /webtoon), 검수(/moderate-image) 등 경로 27개(2026-11-11 라우트 데코레이터 대조). 바깥에 열리지 않고 X-AI-Token이 맞는 요청만 받는다.

- 기본 무료는 gemma4:e4b를 로컬에서 그대로 쓴다
- 강화 무료는 키가 있으면 NVIDIA gemma-4-31b-it, 없으면 이 기기의 gemma4:12b
- 막히면 예비는 gemma4:e4b다. 없는 12b를 찾지 않는다
- 1:1은 로컬·호스팅 토큰을 웹소켓으로 전달한다. 호스팅 SSE와 재작성 교체 흐름은 구현됐고 운영 재배포는 대기 중이다.
- /train은 403 — v1은 학습하지 않는다는 정책

02. 사용 엔진

서버	FastAPI · Pydantic(camelCase 통신)
대화 모델	기본 gemma4:e4b · 강화 gemma-4-31b-it · 예비 gemma4:e4b
그림	diffusers · Animagine XL 4.0 · SD 1.5 (NVIDIA GPU)
프리미엄	Anthropic Claude · xAI Grok (키가 있을 때만)
시험	unittest 92개 · 2026-11-11 기록. 실제 모델 평가는 2026-11-06 e4b 결과이며 호스팅 31B에는 일반화하지 않는다.

03. 상세 기능

- 극소 카드 핵심 + 관련 지식 조각(최대 3개 · 700자)만 넣는 프롬프트 조립
- /ground-check: 답의 개인사가 설정 · 대화에 근거가 있는지 판정
- 월드 진행자: 여러 인물을 번갈아 연기하고 [그림: 이름표]·[이동: 이름 → 장소] 제어 줄을 낸다 → Nest가 검사 후 떼어 냄
- 코디포토넷: 줄거리를 읽고 장면을 먼저 설계(프롬프트) → 그림 → 검수
- 초상 · 장면 그림 검수: 성적 · 미성년자 · 잔혹 · 실존 인물 판정
- /notice-facts로 대화에서 장기 기억 후보를 추출하고, 현재 시간·장소 단서로 상태를 갱신한다.

04. 스크립트 · 평가 연결

학습 대신 평가로 고친다. services/ai/evals 의 스크립트가 실제 모델로 대화를 돌려 기준과 비교하고, 결과는 docs/trainings/ 에 날짜 · 표본과 함께 남긴다.

말투 유지	12/12 (기준 90%)
"너 AI지?" 유도	12/12 (95%)
개인사 단정하지 않기	11/12 (90%)
장면 전환 나레이션	7/12 → 48/48
퀘스트 보상 장면	2/8 → 20/20
퀘스트 판정 잘못 통과	0건
월드 진행자 형식	23/24 (80%)

05. 내부 화면



월드 한 턴의 흐름



웹툰 콘티와 컷

06. 고급 기술

- 판단은 코드로: 작은 모델에게 조건부 지시 대신 '이번 답 첫 줄은 @:' 같은 할 일 하나만 → 나레이션 7/12 → 48/48
- 보상 장면은 코드가 갈래를 나누고 모델은 번호만 고른다 → 2/8 → 20/20
- 호스팅 SSE: 조각 사이 40초·전체 150초 제한. 이미 글이 나온 뒤 실패하면 CPU 예비 답을 이어 붙이지 않는다.
- CPU 예비는 최근 메시지 8개·출력 320토큰으로 줄이고 배경 작업은 예비 전환하지 않는다. 실패 원인은 로그에 남긴다.
- 이 포트폴리오 챗봇은 Rails·Flask·LangGraph·BM25 RAG로 구성되며 11월 8일 Oracle에 배포됐다. 제품과 Ollama를 공유해 요청이 겹치면 지연·503이 발생할 수 있다.

07. 문제점과 개선

31B 품질 미측정	평가는 gemma4:e4b로 맞춰 둠 → 호스팅 31B 재평가 필요(Q08)
데이터 반출	호스팅 API로 대화 · 검수 입력이 나감 → 처리방침 반영 필요(U11)
웹툰 일관성	컷 사이 옷 색이 달라진다(Q04)
장소 이동	월드 진행자가 이동 제어를 거의 쓰지 않는다(0/24)

08. 추가할 서버

- GPU 그림 노드(deploy/ai-node) 분리 운영
- 평가 자동화를 CI의 야간 작업으로
- 임베딩 기반 하이브리드 검색(BM25 + 벡터) 실험

09 · DEVELOPMENT LOG

개발 기록과 운영 현황

2026년 11월 11일 저장소 기준입니다. 코드의 구현, 서버 적용, 과거 측정을 구분하며 아직 확인하지 못한 결과도 남깁니다.

01. 대화 편집과 장기 기억

11월 8일 구현. 마지막 대사의 점 세 개 메뉴에서 수정·삭제·변경사항 추천을 제공합니다. 직접 적은 기억은 매턴 맥락에 넣고, 대화에서 알아챈 사실은 뒤에서 저장합니다. API의 운영 이미지는 최신 제한으로 다시 올리지 않았습니다.

- 시간·장소 단서가 있을 때 현재 상태를 바꿉니다.
- 모델·프롬프트 변경 후 품질 평가 11묶음을 다시 돌리지는 않았습니다.

02. 1대1 응답 스트리밍

11월 9일 로컬 모델의 토큰 전달을 구현했고 11월 11일 호스팅 SSE를 연결했습니다. 웹소켓은 글이 도착하는 대로 보여 주며 모델의 추론 속도 자체를 바꾸지 않습니다.

- 개인사 검사로 다시 쓴 답은 최종 본문으로 교체합니다. 검사 전 문장이 잠깐 보일 수 있습니다.
- 월드 플레이는 아직 완성 후 응답합니다. 운영 API에는 최신 소켓 코드 반영이 필요합니다.

03. 3분 53초 지연 보고와 대응

11월 11일 배포에서 한 턴 약 3분 53초라는 보고가 있었습니다. 호스팅 45초 제한 뒤 긴 입력을 CPU 예비가 다시 처리하는 경로가 원인 후보입니다. 서버 로그로 확정하거나 개선 뒤 운영 시간을 재측정하지는 못했습니다.

호스팅 응답	글 생성은 SSE. 조각 사이 40초, 전체 상한 150초로 구분
CPU 예비	최근 메시지 8개, 출력 320토큰으로 입력·출력 축소
배경 작업	X-AI-Priority로 구분하고 CPU 예비로 보내지 않음
장애 대응	401·402·403·404와 한도·서버 오류에 예비 전환, 실패 원인 로그
서버 반영	api·ai 이미지 재빌드 필요. 적용 전에는 배포 동작이 그대로일 수 있음

04. 포트폴리오 배포 기록

11월 7일 Astro 사이트를 Vercel에 올렸고 11월 8일 Rails·Flask를 Oracle에 연결했습니다. 모델은 제품과 공유하는 Ollama gemma4:e4b 하나입니다.

당시 측정	질문 한 번 약 50초. 평균이나 성능 보장이 아님
공유 자원	Ollama 동시 처리 1개. 제품 대화와 겹쳐 503을 경험
사이트 배포	기존 포트폴리오 프로젝트는 Git 자동 배포 미연결 기록. 푸시와 사이트 재배포를 별도로 확인해야 함

05. 검증 기록과 남은 한계

코드 시험 기록	11월 11일 API 249개, AI 단위 92개
실제 모델 평가	11월 6일 e4b. 나레이션 48/48, 보상 장면 20/20. 31B 재평가 미실시
최근 전체 E2E	미해결 3건: 월드 빈 턴의 상태 코드, 모델 이름, 푸터 약관 링크
미공개 영역	GPU 그림 엔진·SMTP·콘솔 API 연결·프리미엄 키 등은 운영 준비가 남음

06. 근거 문서

- docs/AGENT_HANDOVER.md · docs/STATUS.md · docs/TASKS.md
- docs/trainings/oracle-deploy-latency.md · docs/find_debug/2026-11-11.md
- docs/plan/TESTING.md · docs/trainings/narration-and-reward.md
- docs/web_management/oracle-backend.md

10 · FRAMEWORK NOTES

프레임워크와 역할 분리

제품과 포트폴리오는 목적에 맞춰 다른 프레임워크를 사용합니다. 각 도구를 어디에 썼고, 어떤 경계를 두었는지 실제 코드 구조를 따라 소개합니다.

01. 제품을 구성하는 네 앱

이용자 웹과 운영 콘솔은 별도 Next.js 앱입니다. 둘 다 Nest.js만 호출하며 인증·소유·코레딧·맥락 조립을 Nest에 모읍니다. FastAPI는 전달된 맥락으로 추론하고 DB를 직접 읽지 않습니다.



제품 프레임워크 구성과 신뢰 경계

02. Next.js와 React

apps/web과 console에서 화면, 서버 액션, 인증 쿠키와 API 중계를 구현했습니다. CSS Modules와 공통 디자인 토큰으로 라이트·다크 테마를 유지합니다. 웹과 콘솔은 쿠키와 배포를 분리합니다.

코드	apps/web/app · apps/web/proxy.ts · console/app
선택 이유	대화·공방·운영 화면을 React 컴포넌트로 나누고 서버에서 인증 요청을 중계
경계	무거운 추론을 Next Route에 넣지 않고 Nest에 위임

03. Nest.js와 Prisma

Controller·Guard·Service 경계로 인증·권한·요청 한도와 도메인 동작을 나눕니다. PostgreSQL에는 캠페인 상태·기억·코레딧 장부를 저장하고, DB와 분리할 수 있는 판단은 순수 함수로 시험합니다.

코드	apps/api/src · apps/api/prisma/schema.prisma
선택 이유	여러 화면과 AI가 공유하는 권한·상태·비용 정책을 한곳에서 유지
경계	외부 요청을 검사한 뒤 내부 토큰으로 FastAPI 호출

04. FastAPI와 모델 실행

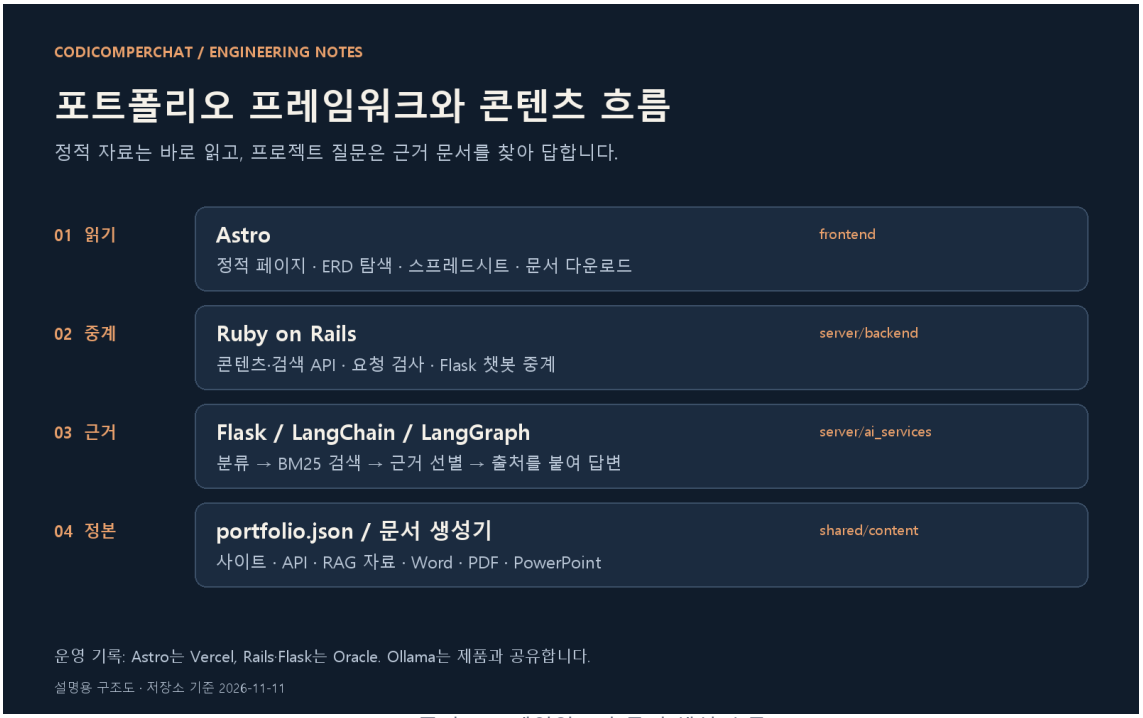
Pydantic 요청 스키마로 추론 입력을 받고 Ollama·호스팅 API·diffusers를 연결합니다. 연기, 요약, 판정, 그림 검수를 나누며 /train은 403으로 닫아 두었습니다.

코드	services/ai/app/routes · services/ai/app/services/llm.py
선택 이유	Python 모델·그림 생태계를 활용하고 도메인 정책과 추론 실행을 분리
경계	제품 웹에서 직접 호출하지 않음. 운영 그림 엔진은 GPU 확보 전 대기

05. Astro와 Rails와 Flask

소개 자료는 Astro로 정적 생성해 모델 응답 없이 읽을 수 있습니다. Rails API는 콘텐츠·검색을 제공하고 Flask 챗봇을 중계합니다. Flask의 LangGraph는 질문 분류·검색·근거 검사·생성을 연결합니다.

Astro	frontend/src/pages · 정적 페이지와 다운로드 파일
Ruby on Rails	server/backend · DB 없는 JSON API, 입력 검사와 챗봇 중계
Flask	server/ai_services · HTTP 인터페이스와 LangGraph 실행
LangChain·LangGraph	문서 분할·BM25 검색·근거 선별·분기·출처 번호



포트폴리오 프레임워크와 문서 생성 흐름

06. 한 콘텐츠에서 여러 결과물로

shared/content/portfolio.json을 정본으로 사이트·Rails 데이터·RAG 자료·Word·PDF·발표자료를 만듭니다. 기획 스프레드시트와 ERD는 각각의 원본을 읽어 뷰어 데이터를 생성합니다.

- 콘텐츠를 바꾼 뒤 build_data.py로 프론트·백엔드·RAG 복사본을 함께 갱신합니다.
- 기존 제품 화면은 실제 캡처를 사용합니다. 새 구조 이미지는 설명용 도식입니다.
- 배포 완료와 코드 구현을 구분해 문서와 챗봇이 같은 한계를 설명하게 합니다.

05 · FUTURE

미래 지향

공개 서비스로 가기 위해 남은 일과, 그다음에 하고 싶은 일을 순서대로 정리했다.

01. 공개 전 반드시

- 11월 11일 api·ai 개선 배포 후 스트리밍과 실제 응답 시간 확인
- 처리방침 국외 이전 표를 실제 구성(NVIDIA · Vercel · Oracle)에 맞추고 법률 검토
- 운영 메일(SMTP) 연결 → 인증 메일
- 운영 콘솔 연결(허용 IP)
- 호스팅 31B 모델로 평가 재실행
- 저장소 비공개 전환

02. 제품

- 월드 플레이도 토큰을 바로 보여 주기
- GPU 서버로 코디포토넷 · 웹툰 공개
- 월드 진행자의 장소 이동 · 긴 플레이 요약
- 세계관 신고 화면

03. 기술

- 하이브리드 검색(BM25 + 임베딩)
- 평가 자동화와 회귀 대시보드
- 관측성(지표 · 추적)
- 여러 대 운영(요청 한도 DB 저장소, AI 노드 분리)